Task 2 - Titanic Dataset Exploratory Data Analysis

Introduction

This project focuses on performing Exploratory Data Analysis (EDA) on the Titanic dataset to identify the factors that influenced passenger survival during the Titanic disaster.

The analysis includes:

- Data Cleaning
- Feature Engineering
- Univariate Analysis
- Bivariate Analysis
- Multivariate Analysis
- Visualization of Survival Patterns

Various visualizations were created to understand how features such as age, gender, passenger class, fare, family size, and embarkation port affected survival outcomes.

Note: Interactive dashboards and advanced visual storytelling were developed separately in Power BI to complement the Python-based analysis.

Dataset Source

Dataset: Titanic Dataset
Source: Kaggle Titanic Machine Learning Competition

Project Objectives

- Understand the structure of the Titanic dataset
- Analyze passenger demographics and survival patterns
- Identify important factors affecting survival
- Perform feature engineering for better analysis
- Create meaningful visualizations for data interpretation
- Extract actionable insights from passenger data

Workflow

1. Data Loading
2. Data Cleaning
3. Handling Missing Values
4. Feature Engineering
5. Exploratory Data Analysis

6. Multivariate Analysis
7. Key Insights & Findings

1. Import Libraries

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("All Library Imported")
```

```
All Library Imported
```

2. Load Dataset

In [3]:
```
# Load the Dateset

train = pd.read_csv(r"D:\Tanushree\prodigy\Task 2\titanic\train.csv")
test = pd.read_csv(r"D:\Tanushree\prodigy\Task 2\titanic\test.csv")

print(f"Train dataset shape: {train.shape}")
print(f"Test dataset shape: {test.shape}")
```

```
Train dataset shape: (891, 12)
Test dataset shape: (418, 11)
```

3. Basic Data Exploration

1. Check headers and data type

In [4]:
```
print(train.columns)
print(train.dtypes)

print(test.columns)
print(test.dtypes)
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
```

```
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
Index(['PassengerId', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch',
       'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
PassengerId      int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
```

2. Check Missing/Null/Blanks

In [4]:

```
print("-------------Missing/Null Values in Train Dataset-----------")
print(train.isnull().sum()[train.isnull().sum() > 0])

blanks_train = (train.astype(str).apply(lambda x: x.str.strip() == '')).sum()

print("----Blank Values for per column----")
print(blanks_train)


print("-------------Missing/Null Values in Test Dataset-----------")
print(test.isnull().sum()[test.isnull().sum() > 0])

blanks_test = (test.astype(str).apply(lambda x: x.str.strip() == '')).sum()

print("----Blank Values for per column----")
print(blanks_test)
```

```
-------------Missing/Null Values in Train Dataset-----------
Age         177
Cabin       687
Embarked      2
dtype: int64
----Blank Values for per column----
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
```

```
Cabin          0
Embarked       0
dtype: int64
-------------Missing/Null Values in Test Dataset-----------
Age       86
Fare       1
Cabin    327
dtype: int64
----Blank Values for per column----
PassengerId    0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked       0
dtype: int64
```

3. Duplicate Values

In [5]:

```
print(f"Duplicate Values in Train Set: {train.duplicated().sum()}")
print(f"Duplicate Values in Test Set: {test.duplicated().sum()}")
```

```
Duplicate Values in Train Set: 0
Duplicate Values in Test Set: 0
```

DATA CLEANING

1. Handling Missing/null Values

In [6]:

```
# Handelling missing
train['Age'] = train.groupby(['Pclass', 'Sex'])['Age'].transform(lambda x: x.fillna(x.me
test['Age'] = test.groupby(['Pclass', 'Sex'])['Age'].transform(lambda x: x.fillna(x.medi

test['Fare'] = test['Fare'].fillna(test['Fare'].median())

train['Cabin'] = train['Cabin'].apply(lambda x: x[0] if pd.notnull(x) else 'U')
test['Cabin'] = test['Cabin'].apply(lambda x: x[0] if pd.notnull(x) else 'U')

train['Embarked'] = train['Embarked'].fillna(train['Embarked'].mode()[0])
```

In [7]:

```
print("-------------Missing/Null Values in Train Dataset-----------")
print(train.isnull().sum()[train.isnull().sum() > 0])

print("-------------Missing/Null Values in Test Dataset-----------")
print(test.isnull().sum()[test.isnull().sum() > 0])
```

```
------------Missing/Null Values in Train Dataset-----------
Series([], dtype: int64)
------------Missing/Null Values in Test Dataset-----------
Series([], dtype: int64)
```

Feature Engineering

Creating New Variables

In [8]:

```
# Suppress the pandas downcasting warning cleanly
pd.set_option('future.no_silent_downcasting', True)
```

1. New Variable: Title Mapping (Train & Test)

In [9]:

```
# Creating Family Size for Title Classification
train['FamilySize'] = train['SibSp'] + train['Parch'] + 1

# Extract Title
train['Title'] = train['Name'].str.extract(r' ([A-Za-z]+)\.', expand=False)

# Group rare titles
title_mapping = {
    'Lady': 'Rare', 'Countess': 'Rare', 'Capt': 'Rare', 'Col': 'Rare',
    'Don': 'Rare', 'Dr': 'Rare', 'Major': 'Rare', 'Rev': 'Rare',
    'Sir': 'Rare', 'Jonkheer': 'Rare', 'Dona': 'Rare',
    'Mlle': 'Miss', 'Ms': 'Miss', 'Mme': 'Mrs'
}

train['Title'] = train['Title'].replace(title_mapping)

print("Titles Extracted for Train")
```

```
Titles Extracted for Train
```

In [10]:

```
# Creating Family Size for Title Classification
test['FamilySize'] = test['SibSp'] + test['Parch'] + 1

# Extract Title
test['Title'] = test['Name'].str.extract(r' ([A-Za-z]+)\.', expand=False)

# Group rare titles
title_mapping = {
    'Lady': 'Rare', 'Countess': 'Rare', 'Capt': 'Rare', 'Col': 'Rare',
    'Don': 'Rare', 'Dr': 'Rare', 'Major': 'Rare', 'Rev': 'Rare',
    'Sir': 'Rare', 'Jonkheer': 'Rare', 'Dona': 'Rare',
    'Mlle': 'Miss', 'Ms': 'Miss', 'Mme': 'Mrs'
}

test['Title'] = test['Title'].replace(title_mapping)

print("Titles Extracted for Test")
```

```
Titles Extracted for Test
```

2. New Variable: Family Group

In [12]:

```
# Recreating FamilySize here for FamilyGroup classification
train['FamilySize'] = train['SibSp'] + train['Parch'] + 1

def fam_group(n):
    if n == 1:
        return 'Solo'
    elif n <= 4:
        return 'Small Family'
    else:
        return 'Large Family'

train['FamilyGroup'] = train['FamilySize'].apply(fam_group)
print("Family Group sorted")
```

```
Family Group sorted
```

3. New Variable: Age Group

In [13]:

```
def age_group(age):
    if age == -1:
        return 'Unknown'
    elif age <= 12:
        return 'Child'
    elif age <= 19:
        return 'Teenager'
    elif age <= 55:
        return 'Adult'
    else:
        return 'Senior'

train['AgeGroup'] = train['Age'].apply(age_group)

print("--- AgeGroup Value Counts ---")
print(train['AgeGroup'].value_counts())
```

```
--- AgeGroup Value Counts ---
AgeGroup
Adult       510
Senior      217
Teenager     95
Child        69
Name: count, dtype: int64
```

4. New Variable: Embarkation Port

In [14]:

```
train['Embarked_Filled'] = train['Embarked'].fillna('S')

port_mapping = {
    'C': 'Cherbourg',
```

```
    'Q': 'Queenstown',
    'S': 'Southampton'
}
train['Embarked_Port'] = train['Embarked_Filled'].map(port_mapping)

print("--- Embarked_Port Value Counts ---")
print(train['Embarked_Port'].value_counts())
```

```
--- Embarked_Port Value Counts ---
Embarked_Port
Southampton    646
Cherbourg      168
Queenstown      77
Name: count, dtype: int64
```

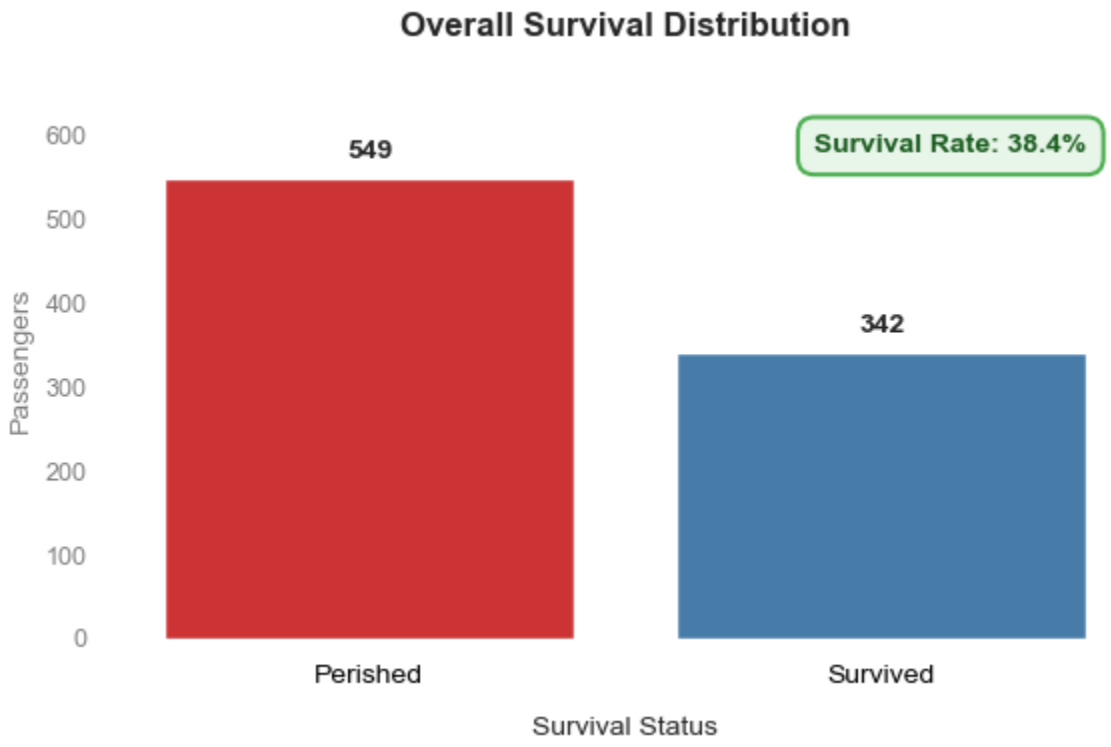
Exploratory Data Analysis (EDA) & Data Visualization

Target Variable Distribution (Univariate Analysis)

In [14]:

```
sns.set_theme(style="white")

plt.figure(figsize=(6, 4))

# Countplot shows the distribution of a single categorical variable
ax = sns.countplot(x='Survived', hue='Survived', data=train, palette='Set1', legend=Fals

# Count labels
for container in ax.containers: ax.bar_label(container, padding=5, fontsize=10, fontweig

# Increase Y-axis top limit
max_count = train['Survived'].value_counts().max()
plt.ylim(0, max_count + 120)

# Survival Percentage
survival_pct = (train['Survived'].sum() / len(train)) * 100

# KPI box
ax.text(0.95, 0.90, f"Survival Rate: {survival_pct:.1f}%",
        transform=ax.transAxes,
        fontsize=10,
        fontweight='bold',
        color='#1b5e20',
        ha='right', va='top',
        bbox=dict(
            boxstyle='round,pad=0.6',
            facecolor='#e8f5e9',
            edgecolor='#4caf50',
            linewidth=1.5
        ))

plt.tick_params(axis='y', colors='grey', labelsize=9)
plt.tick_params(axis='x', colors='black', labelsize=10)
```

```
plt.title('Overall Survival Distribution', fontsize=12, pad=15, fontweight='bold')
plt.xlabel('Survival Status', fontsize=10, labelpad=10)
plt.ylabel('Passengers', fontsize=10, color='grey')

plt.xticks([0,1], ['Perished', 'Survived'])

sns.despine(left=True, right=True, bottom=True)

plt.tight_layout()
plt.show()
```



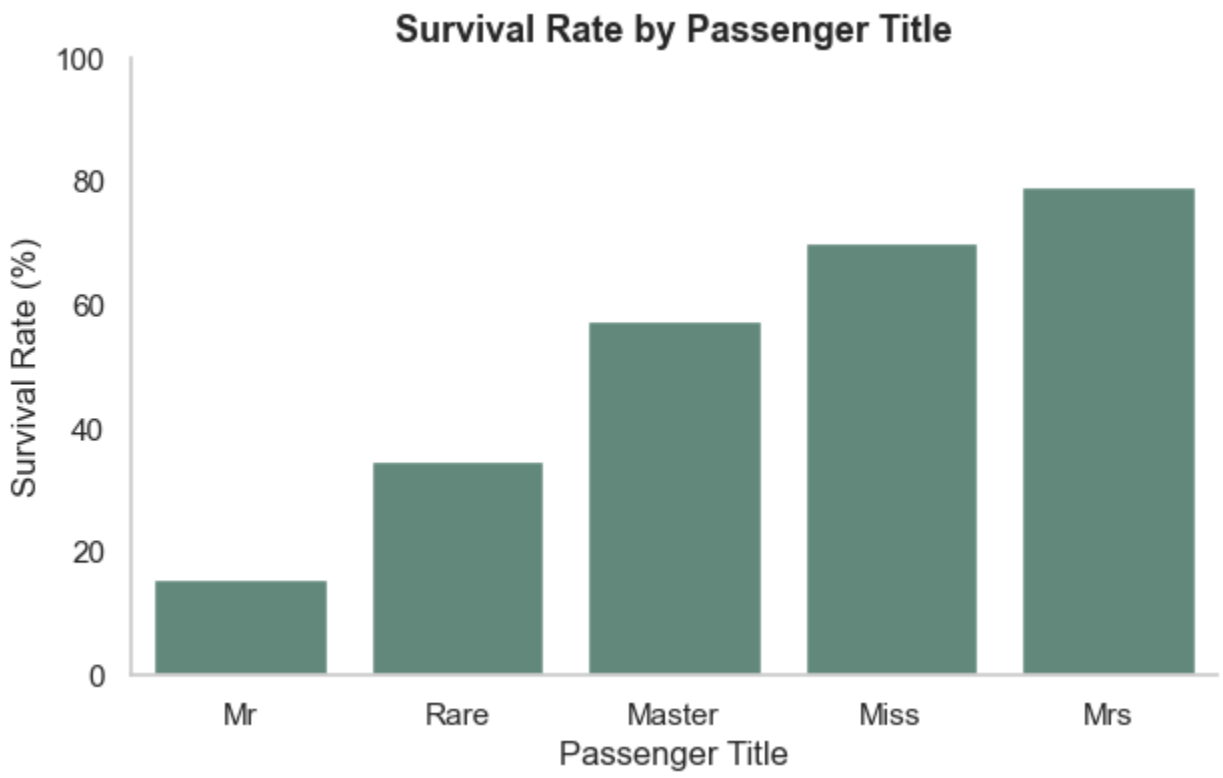
Categorical Relationships with Survival (Bivariate Analysis)

Survival Rate by Passenger Titles

In [18]:

```
plt.figure(figsize=(7,4))

sns.barplot(
    x=title_percentages.index,
    y=title_percentages.values,
    color='#5B8E7D'   # Vintage teal-green
)

plt.ylim(0,100)

plt.ylabel('Survival Rate (%)')
plt.xlabel('Passenger Title')

plt.title(
```

```
    'Survival Rate by Passenger Title',
    fontsize=13,
    fontweight='bold'
)

# Remove gridlines
plt.grid(False)

# Remove top and right borders
sns.despine()

plt.show()
```

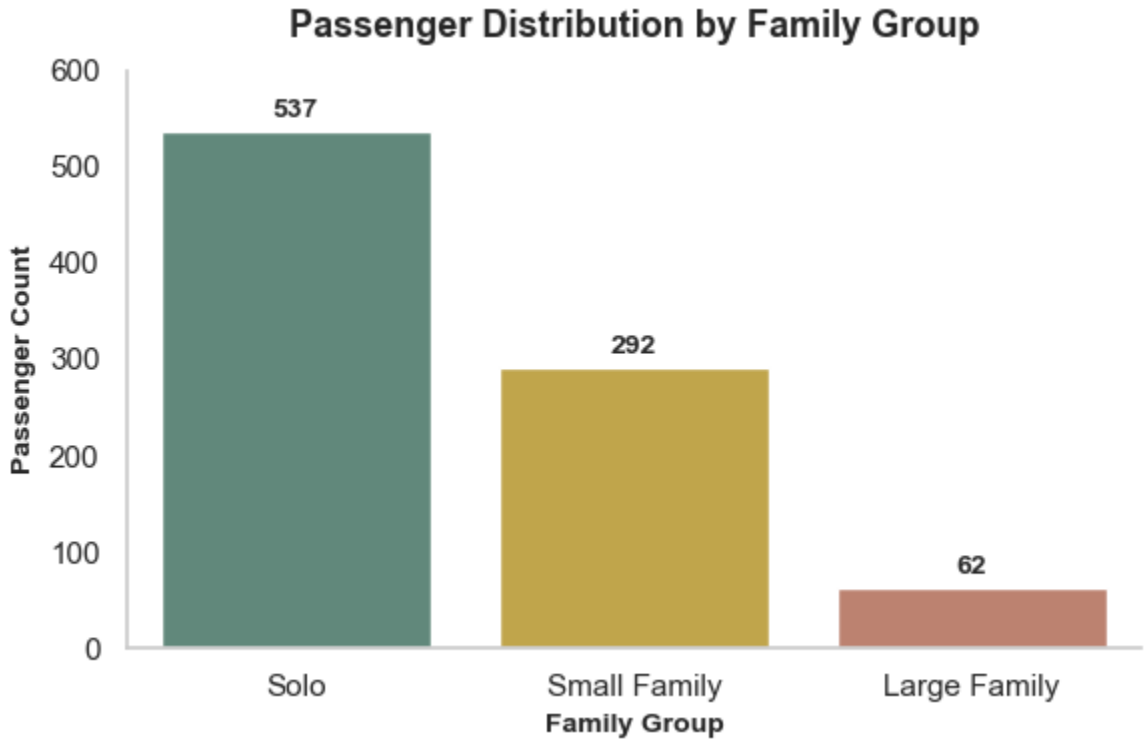


Passenger Distribution by Family Group

In [26]:

```
plt.figure(figsize=(6,4))

ax = sns.countplot(
    data=train,
    x='FamilyGroup',
    hue='FamilyGroup',
    order=['Solo', 'Small Family', 'Large Family'],
    palette=['#D4AF37', '#5B8E7D', '#C97B63'],
    legend=False
)

# Add labels on top of bars
for container in ax.containers:
    ax.bar_label(
        container,
        fmt='%d',
        fontsize=10,
```

```
        fontweight='bold',
        padding=3,
        color='#2C2C2C'
    )

plt.title(
    'Passenger Distribution by Family Group',
    fontsize=13,
    fontweight='bold',
    pad=12
)

plt.xlabel(
    'Family Group',
    fontsize=10,
    fontweight='bold'
)

plt.ylabel(
    'Passenger Count',
    fontsize=10,
    fontweight='bold'
)

# Remove gridlines
plt.grid(False)

# Remove top and right borders
sns.despine()

plt.ylim(0, 600)
plt.tight_layout()
plt.show()
```



Continuous Variables (Age and FamilySize)

Age Distribution

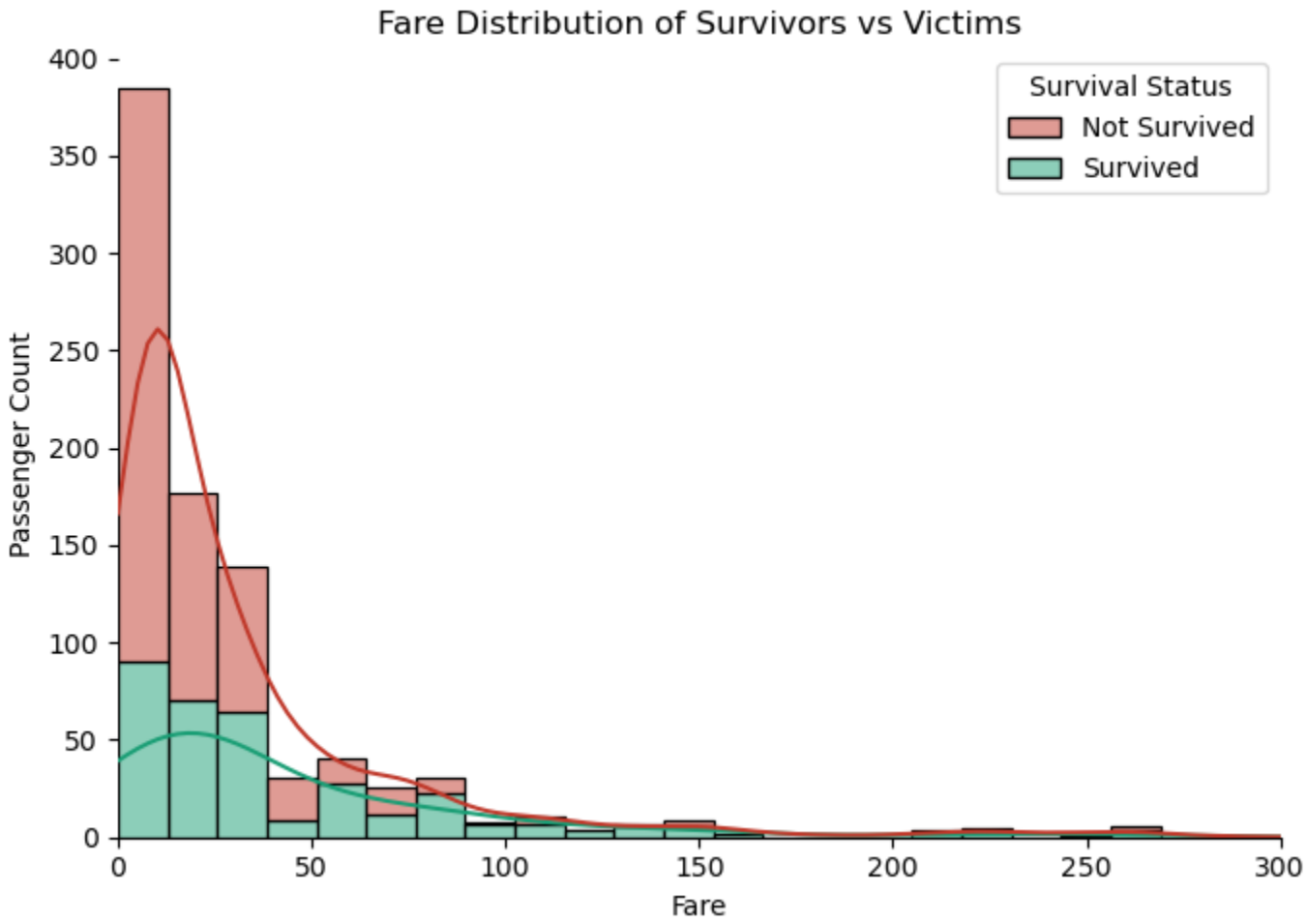
In [5]:

```
plt.figure(figsize=(7, 5))

train['Survival Status'] = train['Survived'].map({0: 'Not Survived', 1: 'Survived'})

sns.histplot(data=train, x='Age', hue='Survival Status', multiple='stack', kde=True, bin

sns.despine(top=True, right=True, left=True, bottom=False)

plt.title('Age Distribution of Survivors vs Victims')
plt.xlabel('Age')
plt.ylabel('Passenger Count')

plt.tight_layout()
plt.show()

# Drop Helper column after use
train.drop(columns=['Survival Status'], inplace=True)
```

Fare Distribution

In [6]:

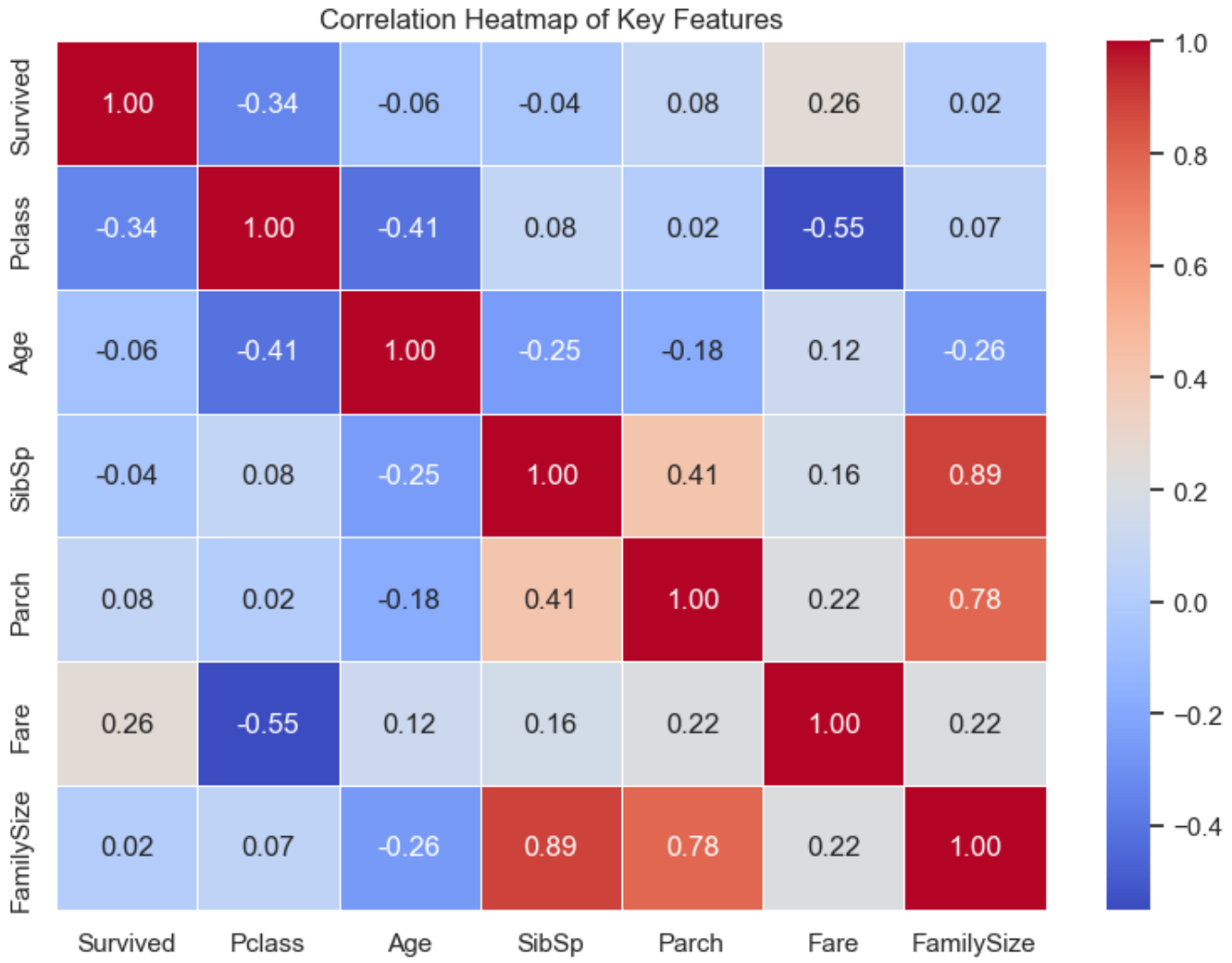
```
plt.figure(figsize=(7, 5))

# Renamed the legends
train['Survival Status'] = train['Survived'].map({0: 'Not Survived', 1: 'Survived'})


sns.histplot(data=train, x='Fare', hue='Survival Status',
             multiple='stack', kde=True, bins=40,
             palette={'Not Survived': '#C0392B', 'Survived': '#1A9E75'})

sns.despine(top=True, right=True, left=True, bottom=False)


plt.title('Fare Distribution of Survivors vs Victims')
plt.xlabel('Fare')
plt.ylabel('Passenger Count')
plt.xlim(0, 300)

plt.tight_layout()
plt.show()

# Drop Helper column after use
train.drop(columns=['Survival Status'], inplace=True)
```



Plot: The Numerical Correlation Heatmap

In [23]:

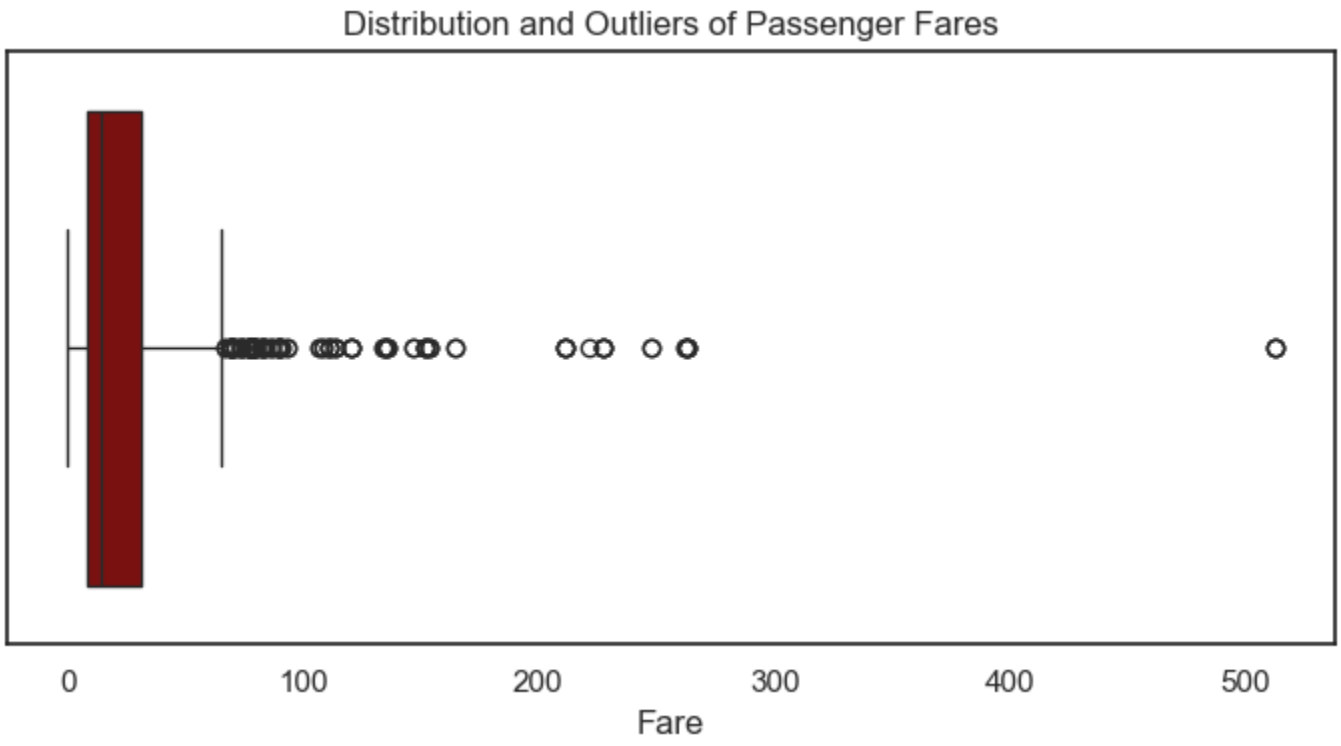
```
plt.figure(figsize=(8, 6))

numerical_feature = train[['Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare', 'Famil

sns.heatmap(numerical_feature.corr(), annot=True, cmap='coolwarm', fmt=".2f", linewidth=

plt.title('Correlation Heatmap of Key Features')
plt.tight_layout()
plt.show()
```



5. Distribution & Outlier Detection (Analyzing 'Fare')

In [24]:

```
plt.figure(figsize=(7, 4))

sns.boxplot(x='Fare', data=train, color='darkred')

plt.title('Distribution and Outliers of Passenger Fares')
plt.xlabel('Fare')

plt.tight_layout()
plt.show()
```



Export from Python

In [25]:

```
print(train.shape)
print(train.info())
print(train.isnull().sum())
print(train.head())
```

```
(891, 19)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 19 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   PassengerId      891 non-null    int64
 1   Survived         891 non-null    int64
 2   Pclass           891 non-null    int64
 3   Name             891 non-null    object
 4   Sex              891 non-null    object
 5   Age              891 non-null    float64
 6   SibSp            891 non-null    int64
 7   Parch            891 non-null    int64
 8   Ticket           891 non-null    object
 9   Fare             891 non-null    float64
 10  Cabin            891 non-null    object
 11  Embarked         891 non-null    object
 12  FamilySize       891 non-null    int64
 13  Title            891 non-null    object
 14  FamilyGroup      891 non-null    object
 15  AgeGroup         891 non-null    object
 16  Embarked_Filled  891 non-null    object
 17  Embarked_Port    891 non-null    object
 18  Survived_pct     891 non-null    int64
dtypes: float64(2), int64(7), object(10)
```

```
memory usage: 132.4+ KB
None
PassengerId        0
Survived           0
Pclass             0
Name               0
Sex                0
Age                0
SibSp              0
Parch              0
Ticket             0
Fare               0
Cabin              0
Embarked           0
FamilySize         0
Title              0
FamilyGroup        0
AgeGroup           0
Embarked_Filled    0
Embarked_Port      0
Survived_pct       0
dtype: int64
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked  FamilySize Title  \
0      0         A/5 21171   7.2500     U        S           2    Mr
1      0          PC 17599  71.2833     C        C           2   Mrs
2      0  STON/O2. 3101282   7.9250     U        S           1  Miss
3      0            113803  53.1000     C        S           2   Mrs
4      0            373450   8.0500     U        S           1    Mr

    FamilyGroup AgeGroup Embarked_Filled Embarked_Port  Survived_pct
0  Small Family    Adult               S   Southampton             0
1  Small Family    Adult               C     Cherbourg           100
2          Solo    Adult               S   Southampton           100
3  Small Family    Adult               S   Southampton           100
4          Solo    Adult               S   Southampton             0
```

In [26]:

```
cols_to_drop = [
    'Survival Status',
    'Survived_pct'
]

train = train.drop(
    columns=[c for c in cols_to_drop if c in train.columns]
)
```

In [27]:

```
print("Rows:", train.shape[0])
print("Columns:", train.shape[1])

print("\nDuplicates:", train.duplicated().sum())

print("\nMissing Values:")
print(train.isnull().sum())
```

```
Rows: 891
Columns: 18

Duplicates: 0

Missing Values:
PassengerId        0
Survived           0
Pclass             0
Name               0
Sex                0
Age                0
SibSp              0
Parch              0
Ticket             0
Fare               0
Cabin              0
Embarked           0
FamilySize         0
Title              0
FamilyGroup        0
AgeGroup           0
Embarked_Filled    0
Embarked_Port      0
dtype: int64
```

In [30]:

```
train.to_csv(
    r"D:\Tanushree\Prodigy\Task 2\Titanic_Tableau_Dataset.csv",
    index=False
)

print("Dataset exported successfully")
```

```
Dataset exported successfully
```

In [32]:

```
import os

print(os.getcwd())
```

Conclusion & Key Insights

1. SURVIVAL RATE

• Only 38.4% of passengers survived. • 549 passengers perished, while 342 survived.

2. GENDER (Strongest Predictor)

• Female survival: 74.2% • Male survival: 18.9% → The "women and children first" policy is clearly reflected.

3. PASSENGER CLASS

• 1st Class: 63.0% survived • 2nd Class: 47.3% survived • 3rd Class: 24.2% survived → Wealth and social status strongly influenced survival.

4. AGE GROUP

• Children (0–12): 58.0% — highest survival • Seniors (60+): 22.7% — lowest survival → Younger passengers received evacuation priority.

5. FAMILY SIZE

• Solo: 30.4% survived • Small Family: 57.9% survived • Large Family: 16.1% survived → Small families had the best survival chances.

6. EMBARKATION PORT

• Cherbourg: 55.4% survival rate • Queenstown: 39.0% survival rate • Southampton: 33.7% survival rate → Passenger composition across ports influenced outcomes.

7. FARE & CLASS CORRELATION

• Pclass negatively correlated with survival (-0.34) • Fare positively correlated with survival (0.26) → Higher fare and class increased survival probability.